ENTERPRISE DEVSECOPS PROJECT SUMMARY

Application Security | CI/CD Security | Kubernetes | GitOps | Supply Chain | Runtime Detection | Security Observability

Field	Details
Candidate	Jagriti Banerjee
Project Type	Private Enterprise DevSecOps and Kubernetes Security Assessment Lab
Assessment Model	Build -> Break -> Detect -> Harden -> Retest -> Report
Primary Focus	Secure SDLC, cloud-native security, software supply-chain integrity, GitOps governance, runtime detection, and observability

Ethics Note	This project is designed as a controlled private-lab portfolio project. The vulnerable application, Kubernetes attack paths, supply-chain demonstrations, and runtime detections are performed only inside a local VM/Kubernetes environment. No third-party production systems are targeted.

Prepared as a professional portfolio summary for security, DevSecOps, cloud security, Kubernetes security, and application security roles.

Table of Contents

1. Executive Summary
2. Project Identity and Business Scenario
3. Assessment Scope and Methodology
4. Repository and Evidence Profile
5. Architecture Overview
6. Application Security Assessment
7. CI/CD Security Pipeline
8. Container and Image Hardening
9. Kubernetes Security Hardening
10. GitOps Deployment with ArgoCD
11. Supply Chain Security and SLSA-Style Evidence
12. Runtime Security and Attack Detection
13. Security Observability with Prometheus and Grafana
14. Key Findings and Risk Summary
15. Compliance and Control Mapping
16. Evidence, Deliverables, and Recruiter Positioning
17. Lessons Learned and Future Roadmap
18. Appendix A. Selected Evidence Screenshots
19. Appendix B. Interview Talking Points and Resume Bullet

1. Executive Summary

This Enterprise DevSecOps Project is a complete private-lab simulation of how a vulnerable application moves through a modern secure software delivery lifecycle. The project starts with an intentionally vulnerable Flask application, validates application-layer weaknesses, builds and scans a hardened container image, deploys secure Kubernetes manifests, adds GitOps control, signs and attests artifacts, simulates runtime attack paths, and visualizes security signals through Prometheus and Grafana.

The project demonstrates practical security engineering capability across application security, DevSecOps automation, Kubernetes hardening, supply-chain security, runtime detection, and professional reporting. It is not only a collection of tools; it is a complete security assessment lifecycle with evidence, remediation, validation, and documentation.

Executive Value Statement	Jagriti Banerjee can present this project as an end-to-end DevSecOps security portfolio asset covering secure CI/CD, container hardening, Kubernetes workload security, GitOps governance, software supply-chain security, runtime threat detection, and security observability.

Dimension	Summary
Business problem simulated	A cloud-native application delivery environment needs to prevent insecure code, vulnerable dependencies, weak containers, misconfigured Kubernetes workloads, unsigned images, and runtime attack paths from reaching production.
Security approach	Shift-left scanning, hardened build, least-privilege deployment, admission control, GitOps drift prevention, image signing, SBOM generation, runtime detection, and security dashboards.
Final outcome	A professional, evidence-backed DevSecOps project demonstrating build, break, detect, harden, retest, and report capabilities.

2. Project Identity and Business Scenario

The project is framed as an enterprise-style DevSecOps and cloud-native security assessment. It models a realistic environment where an organization builds and deploys a web application using CI/CD, Docker containers, Kubernetes workloads, GitOps deployment, and supply-chain metadata. The implemented application is a deliberately vulnerable Flask-based application with SQLite-backed routes and intentionally unsafe code paths for controlled testing.

Project Attribute	Details
Project name	Enterprise DevSecOps Security Lab / Enterprise DevSecOps Red Team Lab
Candidate	Jagriti Banerjee
Assessment style	Private lab-based security assessment and portfolio project
Target environment	Docker, Kind Kubernetes, GitHub Actions, ArgoCD, Gatekeeper, Falco, Prometheus, Grafana
Application target	Intentionally vulnerable Flask application with health, user lookup, ping, and template-rendering endpoints
Security lifecycle	Build -> Break -> Detect -> Harden -> Retest -> Report
Primary audience	Security recruiters, DevSecOps hiring teams, AppSec interviewers, cloud security teams, and consulting-style security reviewers

3. Assessment Scope and Methodology

The assessment scope is limited to the candidate-owned private VM lab and repository artifacts. All attack simulations and validations are performed in a controlled environment. This keeps the project ethical while still demonstrating realistic risk patterns.

In-Scope Area	Coverage
Application security	SQL injection, command injection, unsafe rendering pattern, health endpoint validation, Bandit SAST evidence
Dependency security	pip-audit dependency scan against Python requirements with intentionally vulnerable dependency version
Container security	Multi-stage Dockerfile, non-root runtime user, healthcheck, Trivy image/config/filesystem scans
Kubernetes security	SecurityContext, resource limits, ServiceAccount, least-privilege RBAC, NetworkPolicy, Pod Security restricted labels
GitOps	ArgoCD application, automated sync, self-healing, pruning, drift detection, Git-to-cluster deployment
Supply chain	GHCR image, Cosign signing, public-key verification, Syft SBOM, SBOM attestation, SLSA-style provenance
Runtime detection	Falco alerts for sensitive file access and privileged-pod behavior; RBAC escalation simulation and remediation
Observability	Prometheus, Grafana, Falcosidekick metrics, dashboard and alert rule artifacts

Methodology used in the project:

- Build the application and infrastructure in a private VM lab.

- Introduce controlled vulnerabilities and insecure configurations intentionally.
- Validate exploitability and security impact using safe local commands and Kubernetes simulations.
- Run SAST, dependency, image, Kubernetes manifest, and runtime detection tooling.
- Apply layered remediation through Docker hardening, Kubernetes controls, RBAC, policy-as-code, GitOps, and monitoring.
- Retest the same attack paths and show that controls either detect or block the risk.
- Document the project with professional summaries, evidence references, and recruiter-ready explanations.

4. Repository and Evidence Profile

The uploaded project archive contains a broad evidence set and multiple security-control artifacts. The repository is structured to support portfolio review, interview discussion, and technical validation.

Artifact Category	Observed Count / Examples
Screenshots	159 PNG evidence files in the screenshots folder
Security reports	28 report/evidence files across reports and security/reports
Kubernetes and security manifests	48 YAML/JSON/report artifacts across k8s, monitoring, security, redteam, and gitops folders
CI/CD workflows	1 GitHub Actions workflow file(s)
Documentation	README, SECURITY, executive summary, methodology, findings, evidence index, compliance mapping, and post-mortem documents

This evidence profile is important because hiring teams can quickly see that the project is not just theoretical. It contains commands, reports, YAML manifests, screenshots, and written analysis that map to each major security control.

5. Architecture Overview

At a high level, the project uses a local Ubuntu VM as the main operating environment. Docker provides the container runtime, Kind creates the Kubernetes cluster, GitHub stores source code and CI/CD workflows, and the security stack provides scanning, policy enforcement, GitOps, signing, runtime detection, and dashboards.

Layer	Components
Private VM lab	Ubuntu-Endpoint VM, Docker Engine, kubectl, Helm, Kind
Application layer	Flask API, SQLite demo database, vulnerable test routes, health endpoint
CI/CD layer	GitHub Actions pipeline, Bandit, pip-audit, Trivy, Docker build, GHCR push
Kubernetes layer	Kind cluster, devsecops namespace, deployment, service, ServiceAccount, RBAC, NetworkPolicy
Security governance	OPA Gatekeeper, Pod Security restricted labels, GitOps sync with ArgoCD
Supply chain	GHCR image, Cosign signatures, Syft SBOM, SBOM/provenance attestations
Detection and monitoring	Falco, Falcosidekick, Prometheus, Grafana, custom dashboard and alert rules
Architecture Strength	The architecture is layered intentionally: code security, dependency security, container security, Kubernetes security, GitOps governance, supply-chain integrity, and runtime observability are all represented in one coherent project.

6. Application Security Assessment

The project includes an intentionally vulnerable Flask application used to demonstrate common application-layer weaknesses in a controlled lab. The routes are simple enough to explain clearly in an interview while still representing real vulnerability classes.

Endpoint / Area	Security Issue	Evidence / Value
/user?id=1	SQL injection through string-interpolated query	Demonstrates unsafe query construction and the need for parameterized queries
/ping?host=127.0.0.1	Command injection through shell=True and untrusted input	Demonstrates unsafe shell execution and the need for argument-list execution/input allowlisting
/hello?name=	Unsafe template rendering pattern	Demonstrates server-side rendering risk and template-safety concepts
requirements.txt	Older dependency version for scanner visibility	Creates useful pip-audit evidence for dependency risk
Bandit scan	Static application security testing	Finds risky subprocess usage and other insecure coding patterns

The application security section is useful for interviews because it shows a full chain: identify vulnerable code, validate the behavior safely, run scanning tools, explain impact, and recommend remediation.

7. CI/CD Security Pipeline

The GitHub Actions workflow turns the repository into a security-aware delivery pipeline. It does not only build the application; it performs SAST, dependency scanning, filesystem scanning, image building, image scanning, SARIF/report generation, and container registry publishing.

Pipeline Stage	Tool / Mechanism	Security Purpose
Source checkout	actions/checkout	Fetches repository code for repeatable scanning and build steps
SAST	Bandit	Detects insecure Python code patterns such as risky subprocess usage
SCA	pip-audit	Finds known vulnerable Python dependencies
Filesystem scan	Trivy fs	Checks repository for vulnerabilities, misconfigurations, and secrets where enabled
Container build	Docker	Builds deployable artifact using the project Dockerfile
Image scan	Trivy image	Finds vulnerabilities inside the container image
SARIF/report artifacts	GitHub security and workflow artifacts	Supports auditability and reviewer evidence
Registry push	GitHub Container Registry	Publishes an image artifact for downstream deployment/signing

8. Container and Image Hardening

The Dockerfile demonstrates practical container hardening. The image uses a multi-stage pattern, avoids unnecessary cache, runs under a dedicated non-root user, exposes a healthcheck, and uses Gunicorn instead of the Flask development server.

Control	Implementation	Security Benefit
Multi-stage build	Separate builder and runtime stages	Reduces final image footprint and attack surface
Non-root runtime	Fixed UID/GID 10001	Limits impact if the application process is compromised
No pip cache	pip install --no-cache-dir	Reduces unnecessary artifacts inside the image
Healthcheck	/health endpoint	Allows Docker/Kubernetes to verify service availability
Gunicorn runtime	Production WSGI server	Avoids Flask debug/development server behavior
Image scanning	Trivy image/config/filesystem reports	Provides vulnerability and misconfiguration evidence

9. Kubernetes Security Hardening

The Kubernetes deployment is hardened across identity, privilege, filesystem, resources, health checks, and network boundaries. These controls are important because Kubernetes security failures often come from excessive permissions, default service accounts, privileged pods, missing network boundaries, and weak admission controls.

Security Control	Implementation in Project	Security Outcome
Dedicated namespace	devsecops namespace	Separates app resources from cluster/system resources
ServiceAccount	demo-app-sa	Avoids using the default namespace identity
Token automount disabled	automountServiceAccountToken: false	Prevents unnecessary Kubernetes API token exposure inside the pod
Least-privilege RBAC	Role allows get/list configmaps only	Prevents access to secrets, nodes, or cluster-admin operations
Non-root pod	runAsNonRoot true, runAsUser 10001	Prevents root-level container execution
Privilege escalation disabled	allowPrivilegeEscalation: false	Blocks process privilege escalation
Capabilities dropped	drop ALL	Removes unnecessary Linux capabilities
Read-only root filesystem	readOnlyRootFilesystem: true	Reduces tampering and persistence opportunities
Resource limits	CPU/memory requests and limits	Improves stability and reduces denial-of-service risk
NetworkPolicy	default-deny plus explicit ingress/egress	Enforces zero-trust pod networking
Validation Evidence	The strongest Kubernetes evidence includes RBAC can-i tests, denied secrets access, no cluster-wide access, read-only filesystem proof, NetworkPolicy allowed/blocked pod tests, and Trivy Kubernetes manifest scanning.	

10. GitOps Deployment with ArgoCD

ArgoCD is used to demonstrate GitOps governance. In this model, Git becomes the desired source of truth for Kubernetes manifests. The ArgoCD Application points to the k8s/base path and deploys the app into the devsecops namespace. Automated sync, pruning, retry, and self-healing demonstrate how platform teams reduce manual drift.

GitOps Feature	Project Implementation	Professional Value
Application manifest	gitops/argocd/application.yaml	Defines deployment source, target cluster, namespace, and sync behavior
Automated sync	syncPolicy.automated	Applies Git changes automatically to the cluster
Self-healing	selfHeal: true	Restores live resources if manually changed outside Git
Pruning	prune: true	Removes Kubernetes resources deleted from Git
Drift demo	Manual scale followed by ArgoCD correction	Shows practical governance and auditability

This is an important hiring signal because GitOps is widely used in cloud-native environments. The project demonstrates not only installation, but a useful control story: unauthorized manual changes are detected and corrected back to the approved Git state.

11. Supply Chain Security and SLSA-Style Evidence

The supply-chain phase demonstrates image integrity, SBOM generation, signed attestations, and policy metadata enforcement. The image is pushed to GitHub Container Registry, signed with Cosign, verified with a public key, analyzed with Syft to produce SBOMs, and linked to signed SBOM/provenance evidence.

Supply Chain Control	Artifact / Tool	Purpose
Image registry	GHCR	Stores the container image as a deployable artifact
Image signing	Cosign key pair and signature	Proves that the image was signed by the expected key holder
Signature verification	cosign verify output	Validates image signature using the public key
SBOM generation	Syft SPDX and CycloneDX JSON	Documents software components inside the image
SBOM attachment	cosign attach sbom	Associates the SBOM with the image artifact
SBOM attestation	cosign attest --type spdxjson	Signs the SBOM predicate as evidence
Provenance	SLSA-style JSON predicate	Captures build source, commit, builder identity, and image digest conceptually
Admission metadata	Gatekeeper required labels	Requires deployments to declare owner, SBOM, and signing metadata
Integrity Note	The provenance generated in this private lab should be described honestly as SLSA-style learning evidence. Full SLSA compliance requires a trusted build platform and verifiable build provenance generated by that platform.	

12. Runtime Security and Attack Detection

The runtime security phase uses Falco to detect suspicious activity and uses Kubernetes attack simulations to explain why preventive controls matter. The project demonstrates a privileged pod with hostPath access, sensitive file access detection, and RBAC privilege escalation followed by least-privilege remediation.

Attack / Risk Scenario	Validation	Remediation / Control
Sensitive file access	Falco alert for access pattern	Runtime detection evidence captured in reports/screenshots
Privileged hostPath pod	Pod mounted host root at /host and demonstrated chroot pattern	Pod Security restricted enforcement blocks privileged pod
Overprivileged ServiceAccount	cluster-admin ClusterRoleBinding allowed secrets access and cluster-wide actions	ClusterRoleBinding deleted; namespace RoleBinding grants only ConfigMap read access
RBAC audit	cluster-admin binding enumeration with kubectl/jq	Evidence report documents dangerous binding and after-fix access checks

This phase is valuable because it teaches both sides of security operations: how the attack path works and how to detect/block it. It is a strong demonstration for Kubernetes security, purple team, platform security, and cloud security interviews.

13. Security Observability with Prometheus and Grafana

The monitoring phase deploys Prometheus and Grafana using kube-prometheus-stack and integrates Falco/Falcosidekick metrics for runtime security observability. The project includes custom dashboard JSON and PrometheusRule YAML for Falco detections and event drops.

Observability Component	Project Role
Prometheus	Collects metrics from Kubernetes and Falco/Falcosidekick ServiceMonitors
Grafana	Displays cluster and runtime security dashboards
Falcosidekick	Receives Falco events and exposes metrics for scraping
Custom dashboard	Visualizes Falco detections, events by priority, top rules, kernel event rate, and dropped events
PrometheusRule	Defines alerting logic for Falco detections and dropped events

This section elevates the project from static scanning to operational security. It shows how security events can be converted into metrics, reviewed in dashboards, and connected to alerting logic.

14. Key Findings and Risk Summary

The following table summarizes the most important project findings and control outcomes. Severity values represent private-lab risk modeling rather than production vulnerability disclosure scoring.

ID	Finding	Severity	Status / Outcome
LAB-001	SQL injection in /user route through string-interpolated SQLite query	High	Validated and documented; remediation is parameterized queries
LAB-002	Command injection in /ping route due to shell=True with untrusted input	High	Validated and documented; remediation is argument-list execution and input allowlisting
LAB-003	Old dependency version in requirements.txt	Medium/High	Detected through pip-audit evidence
LAB-004	Need for non-root and hardened container runtime	High	Fixed through Dockerfile and Kubernetes securityContext
LAB-005	Kubernetes workload risk from privileged/hostPath pod pattern	Critical	Detected with Falco and blocked with Pod Security restricted enforcement
LAB-006	Overprivileged ServiceAccount with cluster-admin binding	High/Critical	Demonstrated, then remediated with namespace-scoped least privilege
LAB-007	Potential deployment drift from manual kubectl changes	Medium	Controlled with ArgoCD automated sync and self-healing
LAB-008	Unsigned/unattested image supply-chain gap	Medium/High	Addressed with Cosign signing, Syft SBOM, and attestations
LAB-009	Deployment metadata governance gap	Medium	Gatekeeper blocks deployments missing required supply-chain labels
LAB-010	Runtime detection visibility gap	Medium	Addressed with Falco, Prometheus, Grafana, and PrometheusRule artifacts

15. Compliance and Control Mapping

This project is not a compliance certification; however, it maps clearly to common security control themes used in enterprise assessments. This mapping helps interviewers understand how technical work connects to governance and audit language.

Control Theme	Project Evidence	Framework / Standard Alignment
Secure SDLC	GitHub Actions with SAST, SCA, Trivy, artifact reports	OWASP SAMM / NIST SSDF concepts
Container security	Non-root image, minimal runtime, Trivy scans, healthchecks	CIS Docker / container hardening concepts
Kubernetes workload security	SecurityContext, Pod Security restricted, RBAC, NetworkPolicy	CIS Kubernetes / Kubernetes hardening concepts
Least privilege	ServiceAccount scoping, RoleBinding, blocked secrets access	Zero Trust / IAM least privilege principles
Supply-chain integrity	Cosign signature, Syft SBOM, attestations, provenance	SLSA concepts / software supply-chain governance
Runtime monitoring	Falco detections, Prometheus metrics, Grafana dashboards	Detection engineering / SIEM-style operational visibility
Change control	ArgoCD sync, self-heal, prune, drift correction	GitOps governance / auditability

16. Evidence, Deliverables, and Recruiter Positioning

The project contains a wide evidence base that can be used during interviews, GitHub review, or portfolio walkthroughs. The evidence is organized so reviewers can connect each screenshot or report to a specific technical control.

Deliverable Category	Representative Evidence
Application security	SQL injection proof, command injection proof, Bandit output, pip-audit output
CI/CD security	GitHub Actions workflow, scan job outputs, Trivy job artifacts, GHCR push evidence
Container hardening	Dockerfile, non-root user proof, Trivy image/config scans
Kubernetes hardening	RBAC tests, NetworkPolicy tests, read-only filesystem proof, Pod Security labels
GitOps	ArgoCD Application, resource tree, drift correction, sync health
Supply chain	Cosign signature verification, Syft SBOM, attestations, provenance report
Runtime detection	Falco alert evidence, privileged pod detection, RBAC remediation proof
Observability	Prometheus targets, Falco metrics, Grafana custom dashboard, alert rules
Recruiter Positioning	This project should be positioned as a hands-on DevSecOps and Kubernetes security lab demonstrating end-to-end secure delivery, not as a single-tool tutorial. The strongest interview angle is the lifecycle: build, break, detect, harden, retest, report.

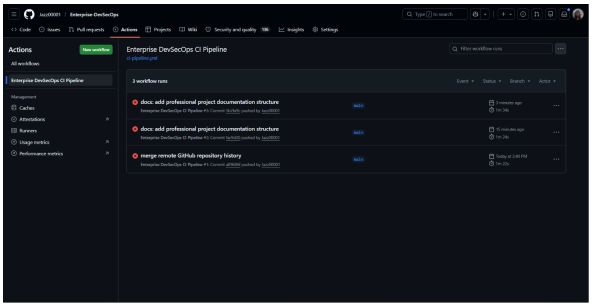
17. Lessons Learned and Future Roadmap

The most important learning from this project is that cloud-native security is layered. No single tool is enough. Real security comes from combining secure coding, scanning, container hardening, Kubernetes controls, GitOps governance, supply-chain metadata, runtime detection, and observability.

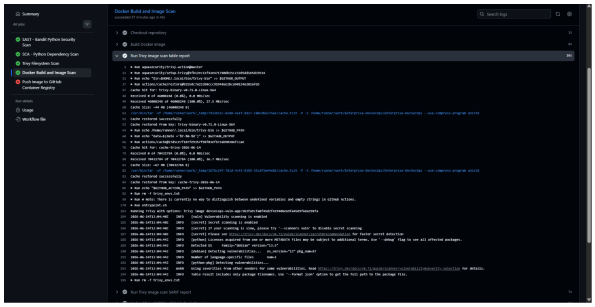
Future Roadmap Item	Value
Move from Kind to a real three-node Kubernetes cluster	Better simulates production-like node and network behavior
Add managed cloud deployment	Demonstrates cloud IAM, managed Kubernetes, and cloud-native security services
Integrate vulnerability management workflow	Connects findings to Jira/Slack remediation workflow and SLAs
Add Kyverno verifyImages or Sigstore Policy Controller	Enforces cryptographic image-signature validation at admission
Improve threat simulation	Adds more runtime attack paths, persistence simulations, and detection tuning
Create short video walkthrough	Improves recruiter understanding and portfolio presentation

Appendix A. Selected Evidence Screenshots

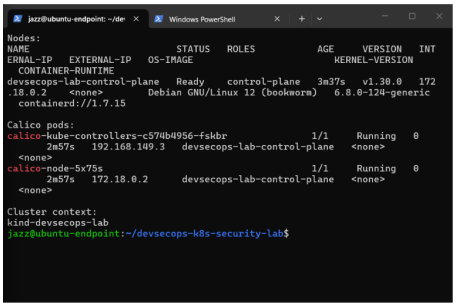
The screenshots below represent selected evidence from the project archive. They are included as sample proof points; the repository contains a larger evidence set.



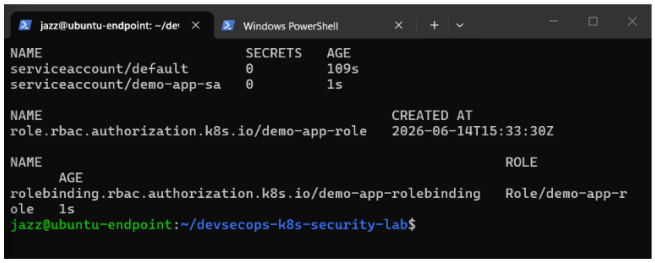
GitHub Actions security pipeline running



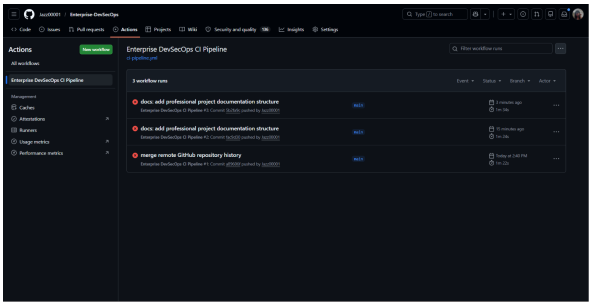
Trivy image scan job evidence



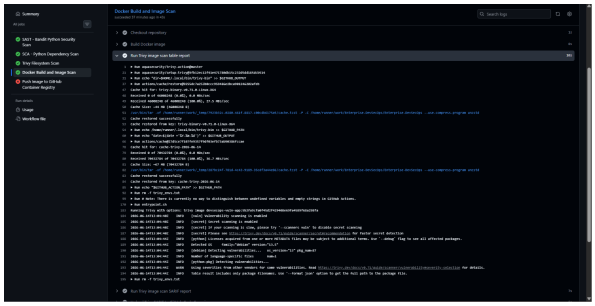
Kind cluster with Calico running



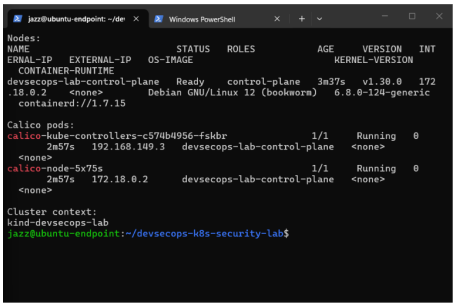
Kubernetes RBAC created



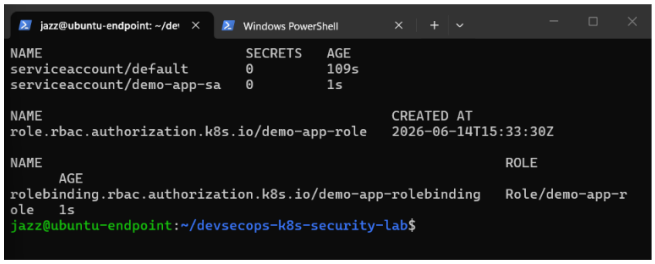
GitHub Actions pipeline running



Trivy scan job



Kind cluster with Calico



RBAC created

Appendix B. Interview Talking Points and Resume Bullet

One-minute project pitch

I built an Enterprise DevSecOps Security Lab in a private VM to simulate how a vulnerable application moves through a secure software delivery lifecycle. I created an intentionally vulnerable Flask app, validated SQL injection and command injection, added CI/CD security scanning with Bandit, pip-audit, and Trivy, hardened the Docker image, deployed the app into Kubernetes with least-privilege RBAC, NetworkPolicies, and Pod Security controls, added GitOps using ArgoCD, signed the image and generated SBOM/provenance evidence with Cosign and Syft, simulated runtime attacks with Falco detection, and visualized security signals using Prometheus and Grafana. The project is documented with evidence, findings, remediation, and recruiter-ready summaries.

Resume bullet

Built an end-to-end Enterprise DevSecOps Security Lab using Docker, Kubernetes Kind, GitHub Actions, ArgoCD, Trivy, Cosign, Syft, OPA Gatekeeper, Falco, Prometheus, and Grafana to simulate vulnerable application testing, secure CI/CD, Kubernetes hardening, supply-chain security, runtime attack detection, RBAC privilege escalation remediation, and security observability.

Technical discussion points

- Explain why shell=True creates command injection risk and how to replace it with argument-list execution.
- Explain why parameterized SQL queries prevent injection compared with string interpolation.
- Explain why non-root containers, dropped capabilities, and read-only filesystems reduce blast radius.
- Explain how Kubernetes ServiceAccounts, Roles, and RoleBindings implement least privilege.
- Explain why privileged pods and hostPath mounts are dangerous in Kubernetes.
- Explain how ArgoCD self-healing prevents unauthorized configuration drift.
- Explain how Cosign, Syft, SBOMs, and attestations support software supply-chain integrity.
- Explain how Falco alerts can be converted into Prometheus metrics and displayed in Grafana dashboards.

Ethical and professional disclaimer

All vulnerabilities and attack simulations in this project were performed only inside a controlled private lab environment owned by the candidate. No public systems, third-party services, or production environments were targeted.